Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 09

Desarrollo de Software

DOCENTE:
Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

1. **Identificar la funcionalidad de React y de su entorno de desarrollo (Node.js, npm y Vite) dentro de una aplicación empresarial.**
2. **Configurar y ejecutar un proyecto de React con Vite e interpretar su estructura (index.html, main.jsx y App.jsx).**
3. **Construir una interfaz con JSX aplicando expresiones, listas con map y key, renderizado condicional, listas anidadas y estilos.**

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la Semana 9 (Introducción a React: qué es React y una SPA, Node.js, npm y Vite, estructura del proyecto, JSX, listas y renderizado condicional) antes del desarrollo del laboratorio.

III. Normas empleadas

- No aplica

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Node.js (versión LTS, que incluye npm), un editor de código (Visual Studio Code), un navegador web y una cuenta de GitHub para el control de versiones.
- La aplicación desarrollada desde la Semana 3 hasta la Semana 8 (incluida la API REST de la Semana 8) y, cuando corresponda, la investigación propia de siete entidades.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Modo de entrega

- El laboratorio se entrega mediante este mismo documento Word, completando cada ejercicio con su respuesta y la justificación solicitada.
- Cada ejercicio que lo requiera debe incluir captura de pantalla del código (App.jsx y App.css), de la página en el navegador y de la terminal cuando se ejecuten comandos.
- El documento debe incluir la dirección (URL) del repositorio de GitHub donde se aloja el proyecto actualizado.
- El documento final se sube por el medio indicado por el docente (aula virtual / plataforma del curso).

VII. Procedimiento

Hasta la Semana 8, cada estudiante construyó en Django una aplicación con CRUD, relaciones, Admin, Templates, consultas avanzadas y una API REST que entrega sus datos en formato JSON. En esta sesión comienza el bloque del front-end: se crea un proyecto de React con Vite, dentro de una carpeta frontend en el mismo repositorio del proyecto Django. Todavía no se consume la API (eso se hace en la Semana 12): los datos se escriben a mano en arreglos con la misma forma que la respuesta de la API de la Semana 8.

El laboratorio se desarrolla en dos partes. En la primera, cada estudiante prepara el entorno, crea el proyecto y construye una interfaz para la aplicación de las Semanas 4 a 8. En la segunda parte,

deberá hacer lo mismo sobre su investigación propia (las siete entidades registradas en el Admin en la Semana 5), cumpliendo los criterios detallados en la sección correspondiente.

El objetivo es que el estudiante comprenda que React no reemplaza a Django: lo complementa. Lo que en un Template se escribía con {{ }}, {% for %} y {% if %}, en React se escribe con llaves, map() y condicionales de JavaScript. En clase la teoría se demuestra con una aplicación de cafetería; cada estudiante la traslada a su propia aplicación con ayuda de la siguiente tabla de equivalencias.

Concepto trabajado en clase	**Equivalente en su aplicación**
Lista de productos (en clase: productos de la cafetería, con id, nombre, precio y stock)	Un arreglo con al menos cinco registros de su entidad principal, con id, un nombre o título, un campo numérico y un campo de estado o cantidad.
Dato que activa un mensaje (en clase: stock igual a 0 muestra Agotado)	Un campo de su entidad que permita mostrar un mensaje distinto (estado, cantidad o disponibilidad).
Relación 1:N anidada (en clase: pedidos con sus detalles)	Su relación 1:N, 1:1 o su modelo intermedio, representada como un arreglo dentro de otro arreglo.
Forma de los datos (en clase: la respuesta de GET /api/productos/)	La respuesta de la API que creó en la Semana 8 para su entidad.

Navegador (http://localhost:5173/)

→ index.html (una sola página)

→ main.jsx (monta React)

→ App.jsx (componente principal)

→ JSX (llaves, map y condicionales)

→ Datos escritos a mano (arreglos)

→ HTML dibujado por React

→ Pantalla

Al finalizar el laboratorio, el estudiante deberá ser capaz de instalar y ejecutar un proyecto de React con Vite, reconocer su estructura y construir una interfaz con JSX que muestre listas, mensajes condicionales, relaciones anidadas y estilos, y publicarla en GitHub.

Enunciados

PARTE 1 — Una interfaz React para la aplicación de las Semanas 4 a 8

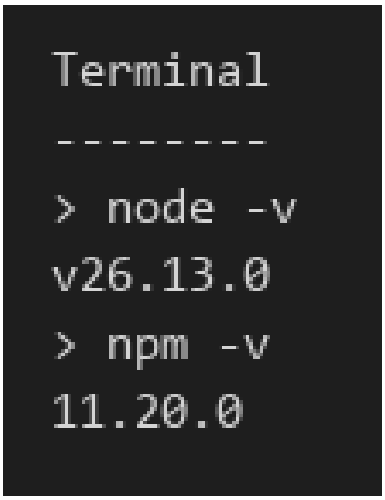
Ejercicio 1 — Verificar el entorno: Node.js y npm

Verifique que Node.js y npm están instalados ejecutando node -v y npm -v en una terminal; si no lo están, instale la versión LTS desde nodejs.org. Documente las versiones obtenidas con una captura. Complete una tabla de equivalencias entre Python y Django, y Node.js y React, para estos elementos: el lenguaje, el gestor de paquetes, el archivo de dependencias, la carpeta de dependencias y el comando que levanta el servidor de desarrollo.

Respuesta:

Node.js y npm instalados. Equivalencias:

Elemento	Python / Django	Node.js / React
Lenguaje	Python	JavaScript (JSX)
Gestor de paquetes	pip	npm
Archivo de dependencias	requirements.txt	package.json
Carpeta de dependencias	venv	node_modules
Servidor de desarrollo	python manage.py runserver	npm run dev



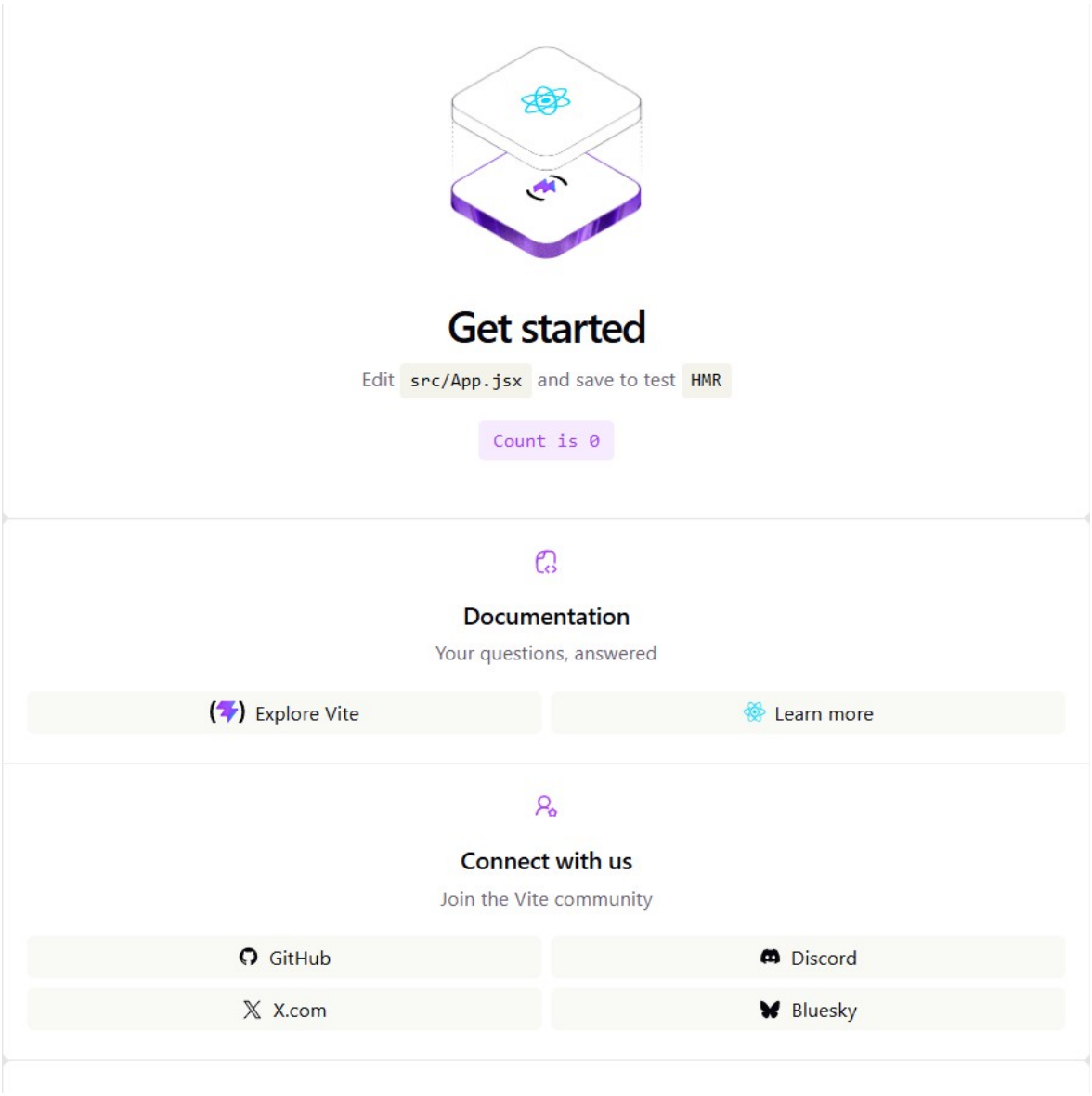
Ejercicio 2 — Crear y ejecutar el proyecto con Vite

Dentro del repositorio de su proyecto Django, cree el proyecto de React con npm create vite@latest frontend -- --template react (si aparece un asistente, elija React y JavaScript). Ejecute npm install y npm run dev, abra http://localhost:5173/ y documente con capturas la salida de la terminal y la página inicial que muestra el navegador.

Respuesta:

Creado con npm create vite@latest frontend -- --template react, luego npm install y npm run dev. Corre en http://localhost:5173/.





Ejercicio 3 — Reconocer la estructura del proyecto

Explore la carpeta frontend y explique con sus palabras el papel de index.html, main.jsx, App.jsx, package.json y node_modules, relacionando cada uno con un elemento que ya conoce de Django (por ejemplo, requirements.txt o manage.py). Documente con una captura la estructura de carpetas.

Respuesta:

Papel de cada archivo y su equivalente en Django:

Archivo	Papel	En Django
index.html	Única página; tiene el div #root	base.html
main.jsx	Monta React en #root	manage.py / wsgi.py
App.jsx	Componente principal con el JSX	Template + View
package.json	Dependencias y scripts	requirements.txt
node_modules	Paquetes instalados	venv

```
frontend
--------
frontend/
├── public/
│   ├── favicon.svg
│   └── icons.svg
├── src/
│   ├── assets/
│   │   ├── hero.png
│   │   ├── react.svg
│   │   └── vite.svg
│   ├── App.css
│   ├── App.jsx
│   ├── index.css
│   ├── main.jsx
│   └── Vet.jsx
├── .gitignore
├── eslint.config.js
├── index.html
├── package-lock.json
├── package.json
├── README.md
└── vite.config.js
```

Ejercicio 4 — Escribir el primer JSX con expresiones

Vacíe el contenido de App.css y de index.css, y reemplace el contenido de App.jsx por un componente que muestre el nombre de su aplicación en un <h1>. Use llaves { } para insertar al menos dos expresiones: una variable y un cálculo o una llamada a una función (por ejemplo, un precio con impuesto usando toFixed(2)). Documente el código y la página en el navegador.

Respuesta:

El componente Vet muestra el título en un h1 y usa llaves para una variable ({aplicacion}) y un cálculo (precio con IGV con toFixed(2)).

src/Vet.jsx

```jsx
function Vet() {
  const aplicacion = 'Veterinaria - Reserva de citas'
  const precioConsulta = 40
  const igv = 0.18

  return (
    <section className="seccion-vet">
      <h1 className="titulo">{aplicacion}</h1>
      <p>Consulta base: S/ {precioConsulta.toFixed(2)}</p>
      <p>Consulta con IGV ({igv * 100}%): S/ {(precioConsulta * (1 + igv)).toFixed(2)}</p>
```

Veterinaria - Reserva de citas

Consulta base: S/ 40.00

Consulta con IGV (18%): S/ 47.20

Insumos

- Jeringa 5ml (unidad) — Stock: 100
- Vacuna antirrabica (dosis) — Stock: 12
- Suero fisiologico (frasco) — Stock: 1 **¡Última unidad!**
- Antiparasitario (tableta) — **Sin stock**
- Gasa esteril (paquete) — Stock: 40

Citas y sus consumos

Firulais (Marta Quispe) — Vacunacion

2026-10-20 09:00 — Dra. Ana Torres — Atendida

Ficha: 12.50 kg — Sano

- Jeringa 5ml × 1 — S/ 1.50
- Vacuna antirrabica × 1 — S/ 25.00

Total de consumos: S/ 26.50

Michi (Jorge Paredes) — Consulta

2026-11-05 10:30 — Dr. Luis Gamarra — Confirmada

Sin ficha clínica

Sin consumos

Total de consumos: S/ 0.00

Rocky (Carla Rios) — Cirugia

2026-11-08 15:00 — Dra. Ana Torres — Pendiente **Por confirmar**

Sin ficha clínica

- Suero fisiologico × 2 — S/ 16.00
- Gasa esteril × 3 — S/ 7.50

Total de consumos: S/ 23.50

Luna (Pedro Soto) — Consulta

2026-11-12 11:15 — Dr. Luis Gamarra — Pendiente **Por confirmar**

Sin ficha clínica

Sin consumos

Total de consumos: S/ 0.00

Ejercicio 5 — Mostrar una lista con map() y key

Cree un arreglo con al menos cinco registros de su entidad principal, con la misma forma que la respuesta de la API de la Semana 8 (los decimales como texto, por ejemplo precio: "6.50"). Muéstrelos en una lista con map() y una key basada en el id, convirtiendo el decimal con Number(...).toFixed(2). Explique con sus palabras qué elemento de los Templates de Django equivale a este map().

Respuesta:

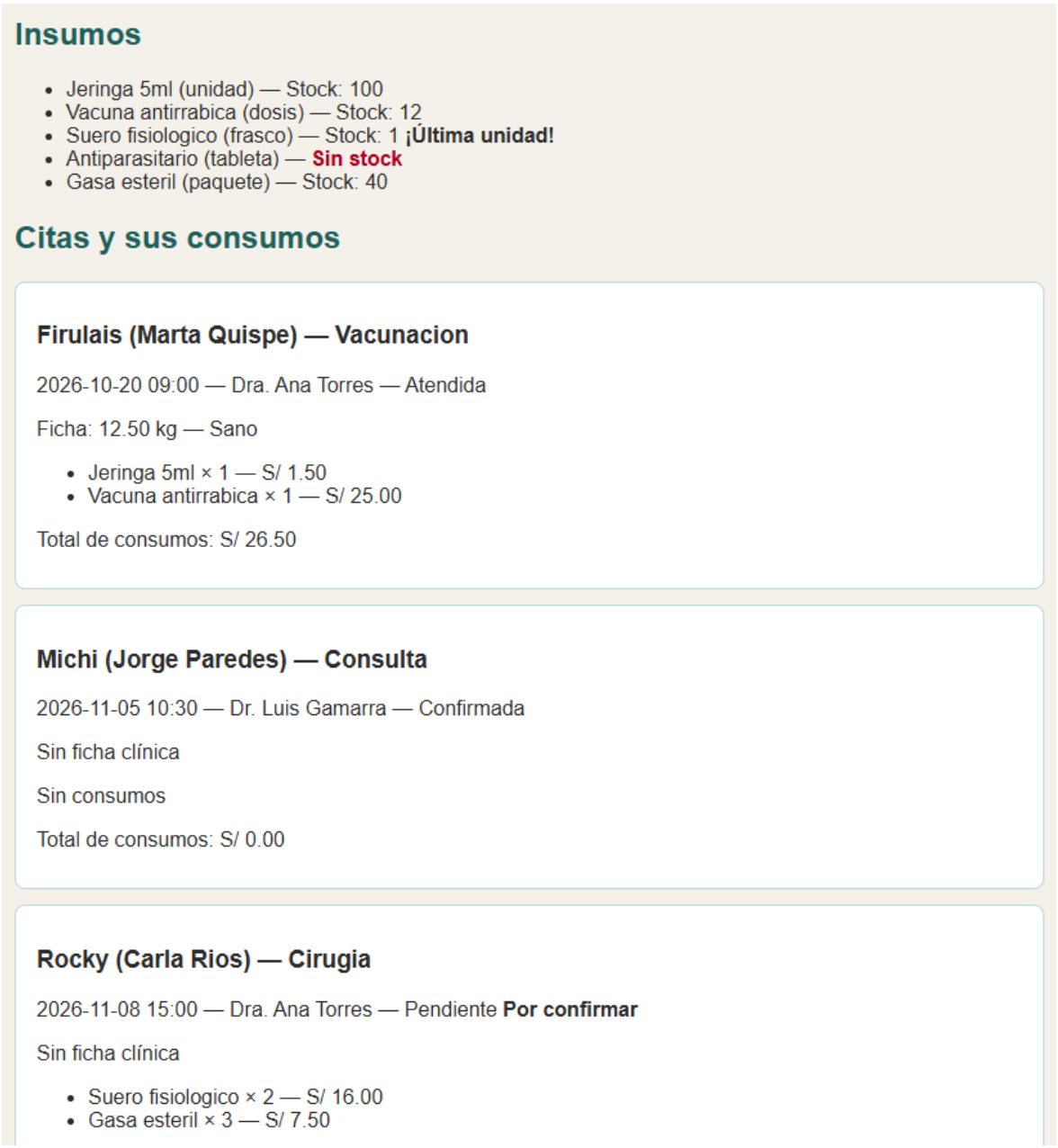
Cinco insumos con la forma de /api/insumos/ (stock numérico). Se muestran con map() y key={insumo.id}. Equivale al {% for %} de los Templates de Django.

src/Vet.jsx

```
  5  const insumos = [
  6    { id: 1, nombre: 'Jeringa 5ml', unidad: 'unidad', stock: 100 },
  7    { id: 2, nombre: 'Vacuna antirrabica', unidad: 'dosis', stock: 12 },
  8    { id: 3, nombre: 'Suero fisiologico', unidad: 'frasco', stock: 1 },
  9    { id: 4, nombre: 'Antiparasitario', unidad: 'tableta', stock: 0 },
 10    { id: 5, nombre: 'Gasa esteril', unidad: 'paquete', stock: 40 },
 11  ]
```

src/Vet.jsx

```
 66            {insumos.map((insumo) => (
 67              <li key={insumo.id}>
 68                {insumo.nombre} ({insumo.unidad}) —{' '}
```



Ejercicio 6 — Aplicar renderizado condicional

Agregue a la lista un mensaje distinto según un campo de su registro, usando un ternario (por ejemplo, Agotado o Stock: n) y otro mensaje con && (por ejemplo, ¡Últimas unidades!). Agregue además un mensaje "No hay registros" que se muestre solo cuando el arreglo esté vacío, y pruébelo vaciando temporalmente el arreglo. Documente el código y ambos resultados.

Respuesta:

Ternario: "Sin stock" o "Stock: n". &&: "¡Última unidad!" si el stock es 1. "No hay registros" si el arreglo está vacío (probado dejando insumos = []).

```
src/Vet.jsx
-----------
 62        {insumos.length === 0 ? (
 63          <p>No hay registros</p>
 64        ) : (
 65          <ul>
 66            {insumos.map((insumo) => (
 67              <li key={insumo.id}>
 68                {insumo.nombre} ({insumo.unidad}) –{' '}
 69                {insumo.stock === 0 ? (
 70                  <span className="sin-stock">Sin stock</span>
 71                ) : (
 72                  `Stock: ${insumo.stock}`
 73                )}
 74                {insumo.stock === 1 && <strong> ¡Última unidad!</strong>}
 75              </li>
 76            ))}
 77          </ul>
 78        )}
```

Insumos

- Jeringa 5ml (unidad) — Stock: 100
- Vacuna antirrabica (dosis) — Stock: 12
- Suero fisiologico (frasco) — Stock: 1 **¡Última unidad!**
- Antiparasitario (tableta) — **Sin stock**
- Gasa esteril (paquete) — Stock: 40

Citas y sus consumos

Firulais (Marta Quispe) — Vacunacion

2026-10-20 09:00 — Dra. Ana Torres — Atendida

Ficha: 12.50 kg — Sano

- Jeringa 5ml × 1 — S/ 1.50
- Vacuna antirrabica × 1 — S/ 25.00

Total de consumos: S/ 26.50

Michi (Jorge Paredes) — Consulta

2026-11-05 10:30 — Dr. Luis Gamarra — Confirmada

Sin ficha clínica

Sin consumos

Total de consumos: S/ 0.00

Rocky (Carla Rios) — Cirugia

2026-11-08 15:00 — Dra. Ana Torres — Pendiente **Por confirmar**

Sin ficha clínica

- Suero fisiologico × 2 — S/ 16.00
- Gasa esteril × 3 — S/ 7.50

Insumos

No hay registros

Ejercicio 7 — Mostrar una relación 1:N con listas anidadas

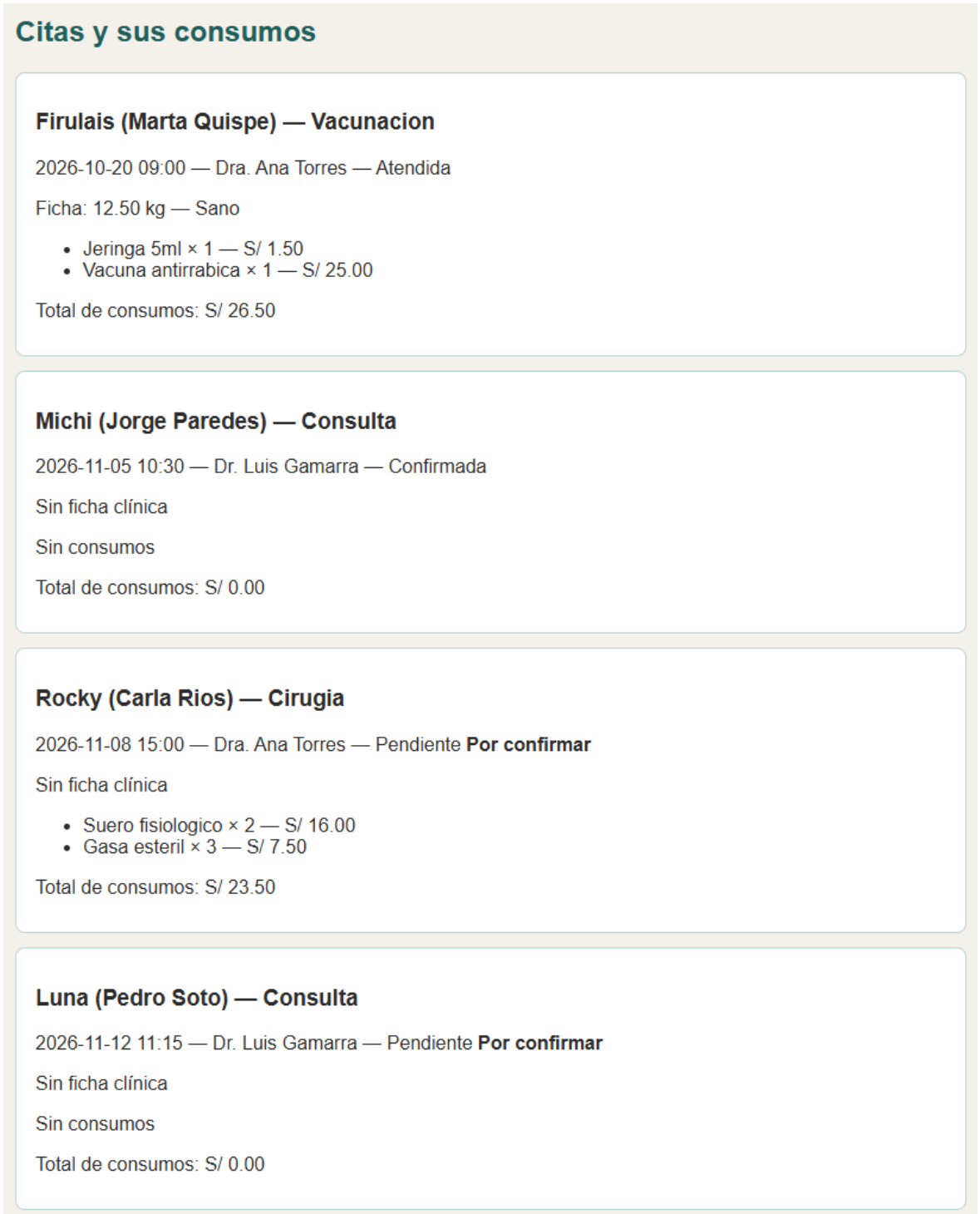
Cree un segundo arreglo que represente una relación 1:N de su aplicación (por ejemplo, pedidos con sus detalles), con un arreglo dentro de cada registro. Muéstrelo con un map() dentro de otro map(), cada uno con su key, y muestre además un total por registro calculado con JavaScript.

Respuesta:

Relación 1:N: cada cita (citas.map) muestra sus consumos (cita.consumos.map), cada uno con su key, y un total calculado con reduce().

src/Vet.jsx

```
----------
 84          citas.map((cita) => (
 85            <div key={cita.id} className="tarjeta-cita">
 86              <h3>
 87                {cita.mascota} ({cita.dueno}) — {cita.servicio}
 88              </h3>
 89              <p>
 90                {cita.fecha} {cita.hora} — {cita.veterinario.nombre} — {cita.estado}
 91                {cita.estado === 'Pendiente' && <strong> Por confirmar</strong>}
 92              </p>
 93              {cita.ficha ? (
 94                <p>
 95                  Ficha: {Number(cita.ficha.peso_kg).toFixed(2)} kg — {cita.ficha.diagnostico}
 96                </p>
 97              ) : (
 98                <p>Sin ficha clínica</p>
 99              )}
100              {cita.consumos.length === 0 ? (
101                <p>Sin consumos</p>
102              ) : (
103                <ul>
104                  {cita.consumos.map((consumo) => (
105                    <li key={consumo.insumo}>
106                      {consumo.insumo} × {consumo.cantidad} — S/ {Number(consumo.subtotal).toFixed(2)}
107                    </li>
108                  ))}
109                </ul>
110              )}
111              <p>
112                Total de consumos: S/{' '}
113                {cita.consumos.reduce((suma, c) => suma + Number(c.subtotal), 0).toFixed(2)}
114              </p>
115            </div>
116          ))
117        )}
118      </section>
```



Ejercicio 8 — Agregar estilos y verificar el flujo completo

Aplique estilos con className y un archivo CSS (al menos tres reglas) y con style={{ }} en al menos un elemento. Cambie un dato de uno de los arreglos y verifique que el navegador se actualiza. Ejecute npm run lint y npm run build y documente la salida. Explique con sus palabras qué diferencia hay entre esta interfaz y los Templates de la Semana 6, y qué falta para conectarla con la API de la Semana 8.

Respuesta:

Estilos con className (App.css, más de tres reglas) y con style={{ }} en los h2. Al cambiar el stock de la vacuna de 12 a 7 el navegador se actualizó solo. npm run lint y npm run build terminaron sin errores.

src/App.css

```
 30  }
 31  .seccion-vet {
 32    margin-bottom: 32px;
 33  }
 34
 35  .seccion-vet .titulo {
 36    color: #1f5f5b;
 37    border-bottom-color: #6bb5ae;
 38  }
 39
 40  .sin-stock {
 41    color: #b00020;
 42    font-weight: bold;
 43  }
 44
 45  .tarjeta-cita {
 46    background-color: #ffffff;
 47    border: 1px solid #b9d9d5;
 48    border-radius: 8px;
 49    padding: 12px 16px;
 50    margin-bottom: 12px;
 51  }
```

src/Vet.jsx

```
 61        <h2 style={{ color: '#1f5f5b' }}>Insumos</h2>
```

Insumos

- Jeringa 5ml (unidad) — Stock: 100
- Vacuna antirrabica (dosis) — Stock: 7
- Suero fisiologico (frasco) — Stock: 1 **¡Última unidad!**
- Antiparasitario (tableta) — **Sin stock**
- Gasa esteril (paquete) — Stock: 40

Citas y sus consumos

Firulais (Marta Quispe) — Vacunacion

2026-10-20 09:00 — Dra. Ana Torres — Atendida

Ficha: 12.50 kg — Sano

- Jeringa 5ml × 1 — S/ 1.50
- Vacuna antirrabica × 1 — S/ 25.00

Total de consumos: S/ 26.50

Michi (Jorge Paredes) — Consulta

2026-11-05 10:30 — Dr. Luis Gamarra — Confirmada

Sin ficha clínica

Sin consumos

Total de consumos: S/ 0.00

Rocky (Carla Rios) — Cirugia

2026-11-08 15:00 — Dra. Ana Torres — Pendiente **Por confirmar**

Sin ficha clínica

- Suero fisiologico × 2 — S/ 16.00
- Gasa esteril × 3 — S/ 7.50

```
npm run lint  /  npm run build
------------------------------
> frontend@0.0.0 lint
> eslint .

> frontend@0.0.0 build
> vite build

[36mvite v8.3.5 [32mbuilding client environment for production...[36m[39m
transforming...
 18 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                  0.47 kB | gzip:  0.30 kB
dist/assets/index-Ckmuafma.css   0.61 kB | gzip:  0.29 kB
dist/assets/index-BbYK3CPD.js  225.82 kB | gzip: 70.66 kB

[32m built in 97ms[39m
```

Diferencia con los Templates de la Semana 6: el Template se arma en el servidor y llega HTML terminado; aquí el navegador construye la interfaz con JavaScript. Falta conectarla con la API de la Semana 8 usando fetch (y habilitar CORS en Django) para reemplazar los datos escritos a mano.

PARTE 2 — Una interfaz React para la investigación propia (registrada en el Admin en la Semana 5)

A partir de aquí trabajará sobre los datos de su investigación propia: las cinco entidades originales, la entidad de la relación OneToOneField y el modelo intermedio de la relación ManyToManyField. El objetivo es construir una interfaz de React para dos entidades de esa investigación y una de sus relaciones, con datos escritos a mano, cumpliendo obligatoriamente el checklist de la siguiente sección antes de continuar.

Ejercicio 9 — Identificar los datos de la investigación propia

Complete la tabla de equivalencias con su investigación: indique qué dos entidades mostrará, qué campo activará el mensaje condicional y qué relación 1:N o modelo intermedio mostrará como lista anidada. Anote también qué respuesta de la API de la Semana 8 usará como modelo para escribir los datos.

Checklist antes de continuar — Parte 2

Criterio	Verificación
1. Node.js y npm están instalados y sus versiones se documentaron.	Captura de node -v y npm -v.
2. El proyecto de Vite está en la carpeta frontend dentro del repositorio y se ejecuta en http://localhost:5173/.	Captura de la terminal y del navegador.
3. App.jsx contiene JSX propio de tu investigación, con un título y al menos dos expresiones entre llaves.	Código de App.jsx + captura.
4. Se muestran al menos dos listas de entidades distintas con map() y key basada en el id.	Código + captura.
5. Hay un renderizado condicional con ternario o && y un mensaje de lista vacía.	Código + captura del mensaje.

6. Una relación 1:N o el modelo intermedio se muestra con listas anidadas.	Código + captura.
7. Los estilos se aplican con className y un archivo CSS propio.	Código de App.css + captura.
8. La consola del navegador no muestra advertencias y npm run lint y npm run build terminan sin errores.	Capturas de la consola y de la terminal.
9. Se explica por escrito qué diferencia hay con los Templates de Django y qué falta para consumir la API.	Párrafo de justificación.

Ejercicio 10 — JSX y listas de la investigación propia

Escriba en App.jsx los arreglos de dos entidades de su investigación, con la forma de la respuesta de su API, y muéstrelos con JSX, expresiones entre llaves, map() y key basada en el id.

Ejercicio 11 — Renderizado condicional y estilos en la investigación propia

Agregue el mensaje condicional (con ternario y con &&) y el mensaje de lista vacía. Aplique estilos con className y un archivo CSS propio, y con style={{ }} en al menos un elemento.

Ejercicio 12 — Relación anidada en la investigación propia

Muestre la relación 1:N o el modelo intermedio de su investigación con listas anidadas, con un total o un conteo por registro calculado con JavaScript.

Ejercicio 13 — Verificar la interfaz de la investigación propia

Verifique que la consola del navegador no muestra advertencias (por ejemplo, de key), ejecute npm run lint y npm run build, y documente la salida. Compruebe que las carpetas node_modules y dist no están en el repositorio.

Ejercicio 14 — Publicar en GitHub

Actualice el README.md describiendo cómo instalar y ejecutar el frontend (npm install y npm run dev) y qué muestra la interfaz. Haga commit y push del proyecto de React, sin las carpetas node_modules ni dist. Incluya en este documento la dirección (URL) del repositorio, conforme a lo indicado en Modo de entrega.

Entregables

- Cada enunciado debe tener su respuesta, con la justificación y la captura de pantalla solicitada (especialmente el código de App.jsx y App.css, la página en el navegador y la salida de npm run lint y npm run build).
- Repositorio en GitHub actualizado, con la carpeta frontend, el README.md y el código correspondiente, sin node_modules ni dist.
- Dirección (URL) del repositorio de GitHub incluida explícitamente en este documento, conforme a lo indicado en Modo de entrega.

Repositorio: https://github.com/RXDG2908/DesAplEmp-S02

Conclusiones:

React construye la interfaz con JSX, expresiones entre llaves y map() con key.

El renderizado condicional (ternario y &&) reemplaza al {% if %} de Django.

Las listas anidadas muestran relaciones 1:N, como cita y consumos.

Con datos escritos a mano se prueba la interfaz antes de conectarla a la API.

Con React el navegador arma la pantalla con JavaScript, mientras que con los Templates el servidor entrega el HTML ya terminado.
Node.js y npm cumplen el mismo papel que Python y pip: package.json equivale a requirements.txt y node_modules al entorno virtual.
JSX permite mezclar la estructura de la página con valores y cálculos escritos entre llaves, como el precio de la consulta con IGV.
Recorrer una lista con map() y darle a cada elemento una key es el equivalente al for de los Templates de Django.
El renderizado condicional muestra mensajes como Sin stock o Sin ficha clínica según el dato, sin escribir varias versiones de la página.
La relación entre citas y consumos se muestra con un recorrido dentro de otro, siguiendo la misma forma del JSON de la API.
Todavía no se consume la API: los datos están escritos a mano y el siguiente paso es pedirlos al servidor.